Example:
Test Case 3: Purchase Officer Request Review Test

Code

```
import os
import sys
import pathlib
import time
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options

# Paths and configuration
BASE_DIR = pathlib.Path(__file__).resolve().parent.parent
if str(BASE_DIR) not in sys.path:
    sys.path.insert(0, str(BASE_DIR))

SCREENSHOT_DIR = BASE_DIR / "test-results" / "screenshots"
SCREENSHOT_DIR.mkdir(parents=True, exist_ok=True)

# Test Configuration - configurable via environment variables
BASE_URL = os.environ.get("BASE_URL", "http://localhost:3000")
LOGIN_URL = f"{BASE_URL}/login"
PO_REQUISITIONS_URL = f"{BASE_URL}/purchase-officer/requisitions"
PO_USERNAME = os.environ.get("TEST_PO_USERNAME", "test_po")
PO_PASSWORD = os.environ.get("TEST_PO_PASSWORD", "TestPO@2026")
HEADLESS_MODE = os.environ.get("HEADLESS", "true").lower() in ("true", "1", "yes")


def setup_test_requisition_for_po():
    """Ensure a valid test requisition in SUBMITTED state is assigned to test_po."""
    import django
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'backend.settings')
    django.setup()
    from accounts.models import User, Department, Role
    from procurement.models import Requisition, RequisitionItem

    po_user = User.objects.get(username=PO_USERNAME)
    staff_user = User.objects.filter(role__name='Department Staff').first()
    dept = Department.objects.filter(name='Medical & Clinical Services').first() or Department.objects.first()

    # Look for an existing submitted requisition assigned to test_po
    req = Requisition.objects.filter(assigned_purchase_officer=po_user, status='SUBMITTED').first()
    if not req:
        # Check any submitted requisition and assign to test_po
        req = Requisition.objects.filter(status='SUBMITTED').first()
        if req:
            req.assigned_purchase_officer = po_user
            req.save()
        else:
            # Create a clean test requisition in SUBMITTED status
            count = Requisition.objects.count() + 1
            req = Requisition.objects.create(
                req_number=f"REQ-2026-{count:05d}",
                title="Automated Test - Patient Monitor Review",
                priority="HIGH",
                justification="Urgent clinical requirement for Operation Theatre monitoring.",
                department=dept,
                requested_by=staff_user,
                assigned_purchase_officer=po_user,
                status="SUBMITTED",
                estimated_budget=45000.00
            )
            RequisitionItem.objects.create(
                requisition=req,
                item_name="Multiparameter Patient Monitor",
                category="Biomedical Equipment",
                quantity=2,
```

```
                estimated_unit_price=22500.00,
                specifications="High-resolution vitals monitor with ECG, SpO2, and NIBP."
            )
    return req


class TestPurchaseOfficerReview:
    """
    Test Case 3: Purchase Officer Request Review Test Suite
    Validates reviewing submitted requisitions, routing decisions, remarks,
    and workflow status transitions.
    """

    @classmethod
    def setup_class(cls):
        """Prepare test requisition in database for test_po."""
        cls.target_req = setup_test_requisition_for_po()

    def setup_method(self, method):
        """Setup Chrome WebDriver before each test execution."""
        options = Options()
        if HEADLESS_MODE:
            options.add_argument("--headless=new")
        options.add_argument("--window-size=1920,1080")
        options.add_argument("--no-sandbox")
        options.add_argument("--disable-dev-shm-usage")
        options.add_argument("--disable-gpu")
        options.add_argument("--ignore-certificate-errors")

        self.driver = webdriver.Chrome(options=options)
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 12)

    def teardown_method(self, method):
        """Tear down and close the WebDriver session after each test."""
        if hasattr(self, "driver") and self.driver:
            self.driver.quit()

    def click_element(self, element):
        """Helper to reliably scroll and click element across animations."""
        self.driver.execute_script("arguments[0].scrollIntoView({block: 'center'});", element)
        try:
            element.click()
        except Exception:
            self.driver.execute_script("arguments[0].click();", element)

    def login_as_purchase_officer(self):
        """Helper to authenticate as Purchase Officer."""
        driver = self.driver
        wait = self.wait

        driver.get(LOGIN_URL)
        username_field = wait.until(EC.visibility_of_element_located((By.NAME, "username")))
        username_field.clear()
        username_field.send_keys(PO_USERNAME)

        password_field = wait.until(EC.visibility_of_element_located((By.NAME, "password")))
        password_field.clear()
        password_field.send_keys(PO_PASSWORD)

        login_btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']")))
        self.click_element(login_btn)

        wait.until(EC.url_contains("/purchase-officer/dashboard"))

    def test_purchase_officer_review_routing_validation(self):
        """
        Negative Test: Attempt proceeding with review without selecting Yes/No
        for Technical Evaluation decision. Verify proper validation error banner appears.
        """
        driver = self.driver
        wait = self.wait

        self.login_as_purchase_officer()
```

```
        driver.get(PO_REQUISITIONS_URL)
        wait.until(EC.url_contains("/purchase-officer/requisitions"))

        wait.until(EC.visibility_of_element_located((By.TAG_NAME, "table")))

        # Locate requisition row and open review modal
        req_row = wait.until(
            EC.visibility_of_element_located((By.XPATH, f"//tr[contains(., '{self.target_req.req_number}')]"))
        )
        review_btn = req_row.find_element(By.XPATH, ".//button[contains(., 'Review') or contains(., 'View')]")
        self.click_element(review_btn)

        # Wait for modal
        wait.until(EC.visibility_of_element_located((By.XPATH, f"//h2[contains(., '{self.target_req.title}')]")))

        # Click proceed without making routing decision
        proceed_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[.//span[text()='Proceed']]")))
        self.click_element(proceed_btn)

        err_banner = wait.until(
            EC.visibility_of_element_located((By.XPATH, "//div[contains(@class, 'bg-rose-50')]"))
        )
        driver.save_screenshot(str(SCREENSHOT_DIR / "tc3_routing_validation_error.png"))

        assert "Please select whether Technical Evaluation is required" in err_banner.text, (
            f"Expected routing decision error, got: {err_banner.text}"
        )

        # Close modal
        close_btn = driver.find_element(By.XPATH, "//div[contains(@class, 'fixed')]//button[contains(., 'Close') or ./*[name()='svg'
        self.click_element(close_btn)

    def test_purchase_officer_review_proceed_success(self):
        """
        Positive Test: Complete review of submitted purchase requisition.
        Verify:
        1. Login as Purchase Officer and navigate to Requisitions Review.
        2. Select pending requisition and open review modal.
        3. Review item details and system advisory.
        4. Select Technical Evaluation decision YES.
        5. Enter verification remarks and confirm Proceed action.
        6. Verify success confirmation notification.
        7. Verify updated status in table (Pending Technical Evaluation).
        """
        driver = self.driver
        wait = self.wait

        # Step 1: Login
        self.login_as_purchase_officer()
        driver.save_screenshot(str(SCREENSHOT_DIR / "tc3_step1_po_dashboard.png"))

        # Step 2: Open Requisitions Review table
        driver.get(PO_REQUISITIONS_URL)
        wait.until(EC.url_contains("/purchase-officer/requisitions"))
        wait.until(EC.visibility_of_element_located((By.TAG_NAME, "table")))
        driver.save_screenshot(str(SCREENSHOT_DIR / "tc3_step2_requisitions_table.png"))

        # Step 3: Locate target requisition row and click Review
        req_row = wait.until(
            EC.visibility_of_element_located((By.XPATH, f"//tr[contains(., '{self.target_req.req_number}')]"))
        )
        review_btn = req_row.find_element(By.XPATH, ".//button[contains(., 'Review') or contains(., 'View')]")
        self.click_element(review_btn)

        # Step 4: Verify details in review modal
        modal_title = wait.until(
            EC.visibility_of_element_located((By.XPATH, f"//h2[contains(., '{self.target_req.title}')]"))
        )
        assert modal_title.is_displayed(), "Requisition title not displayed in modal"
        driver.save_screenshot(str(SCREENSHOT_DIR / "tc3_step3_review_modal_details.png"))

        # Step 5: Select Technical Evaluation YES
        yes_radio = wait.until(EC.presence_of_element_located((By.ID, "po_tech_decision_yes")))
        self.click_element(yes_radio)
```

```
driver.save_screenshot(str(SCREENSHOT_DIR / "tc3_step4_technical_decision_selected.png"))

# Step 6: Click Proceed to open confirmation panel
proceed_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[.//span[text()='Proceed']]")))
self.click_element(proceed_btn)

# Enter remarks
remarks_input = wait.until(
    EC.visibility_of_element_located((By.XPATH, "//textarea[contains(@placeholder, 'Specifications verified') or contains(@p
)
remarks_input.clear()
remarks_input.send_keys("Technical specifications and clinical budget verified. Proceeding for Technical Officer evaluation.
driver.save_screenshot(str(SCREENSHOT_DIR / "tc3_step5_confirm_remarks_entered.png"))

# Step 7: Confirm Proceed action
confirm_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Confirm & Proceed')]")))
self.click_element(confirm_btn)

# Step 8: Verify Success Notification
success_banner = wait.until(
    EC.visibility_of_element_located((By.XPATH, "//div[contains(@class, 'bg-emerald-50')]"))
)
assert "proceeded" in success_banner.text.lower(), f"Expected proceed confirmation, got: {success_banner.text}"
driver.save_screenshot(str(SCREENSHOT_DIR / "tc3_step6_success_confirmation.png"))

# Step 9: Switch to Proceeded tab and verify status in table
proceeded_tab = driver.find_element(By.XPATH, "//button[contains(., 'Proceeded')]")
self.click_element(proceeded_tab)
time.sleep(1)

updated_row = wait.until(
    EC.visibility_of_element_located((By.XPATH, f"//tr[contains(., '{self.target_req.req_number}')]"))
)
driver.save_screenshot(str(SCREENSHOT_DIR / "tc3_step7_updated_status_table.png"))

assert "Pending Technical Evaluation" in updated_row.text, (
    f"Expected status 'Pending Technical Evaluation', found: {updated_row.text}"
)
```

Screenshot

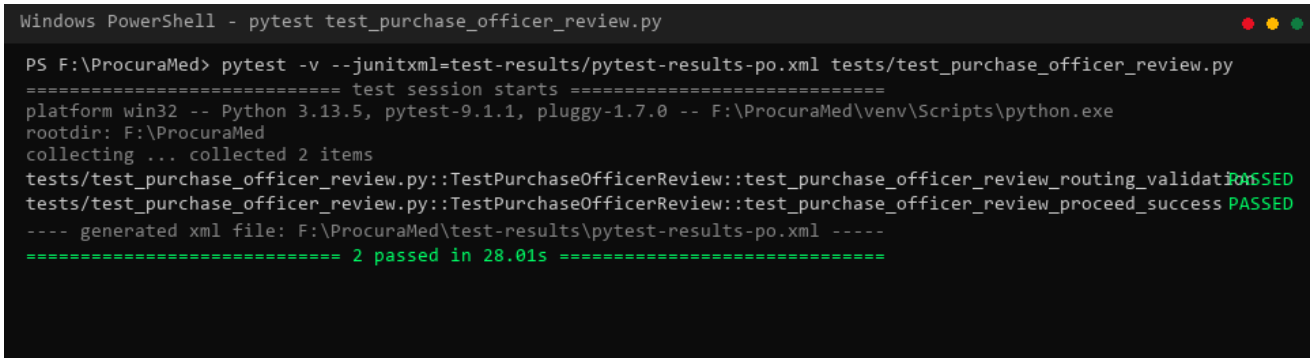


Figure 1 : Purchase Officer Request Review Test successfully (Pytest Execution Results)

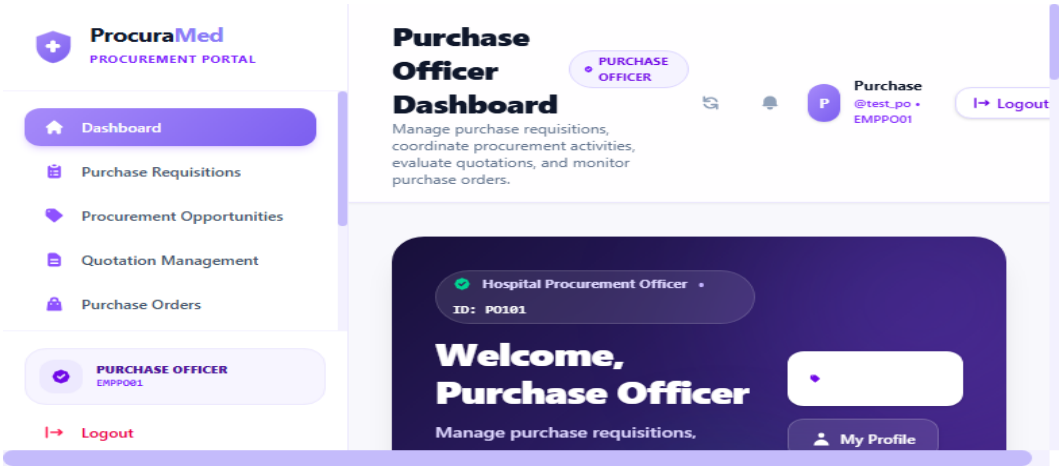


Figure 2 : Purchase Officer Dashboard with Overview Metrics

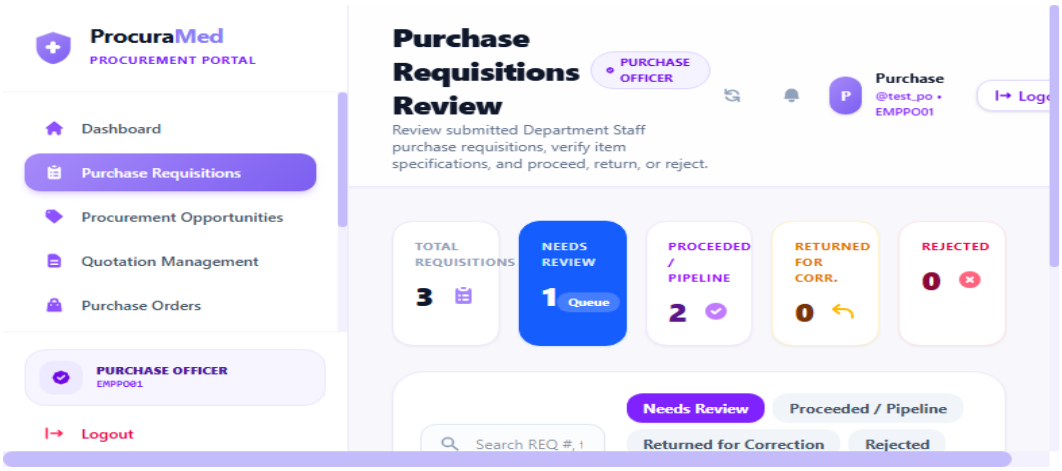


Figure 3 : Submitted Purchase Requests Assigned to Purchase Officer



Figure 4 : Review Modal displaying Requisition Details, Line Items, and Clinical Justification

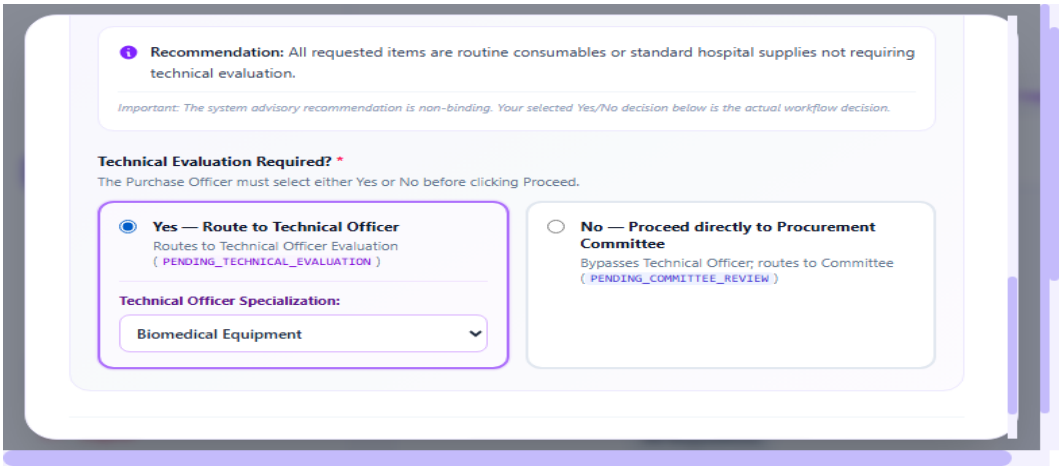


Figure 5 : Technical Evaluation Decision Selected (Yes - Biomedical Equipment)

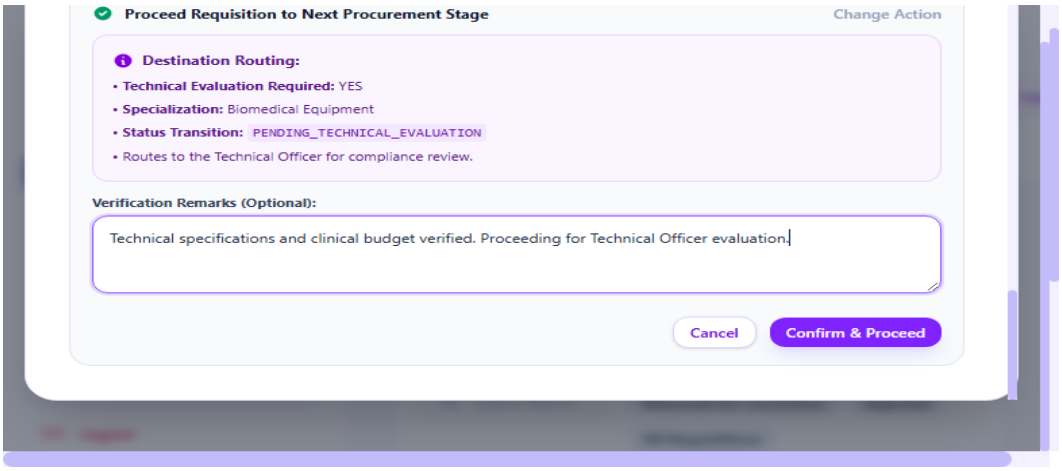


Figure 6 : Purchase Officer Verification Remarks Entered in Action Confirmation Panel

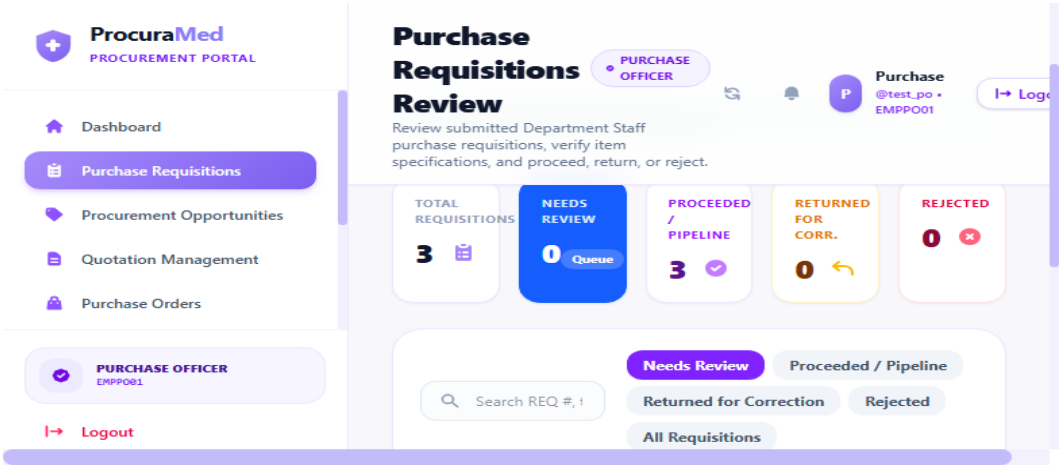


Figure 7 : Requisition Proceeded Success Notification Banner

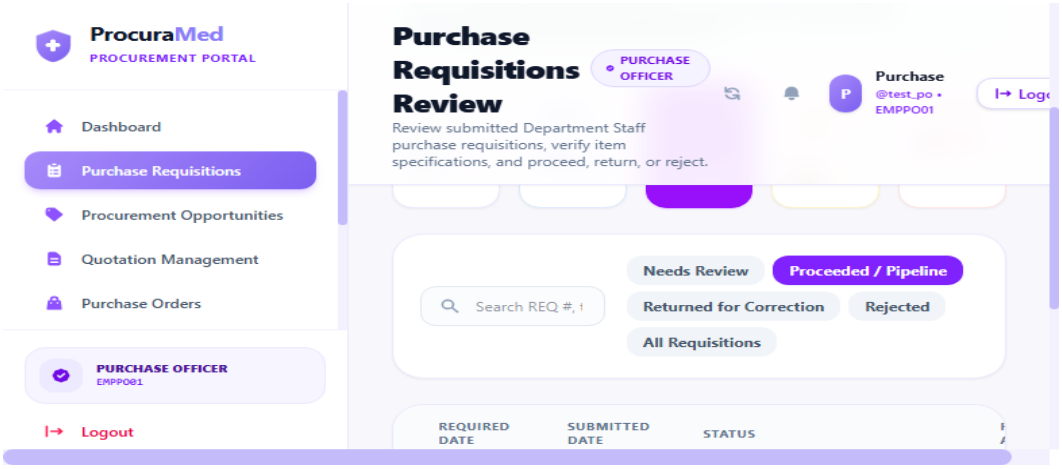


Figure 8 : Verified Requisition Status 'Pending Technical Evaluation' in Proceeded Table

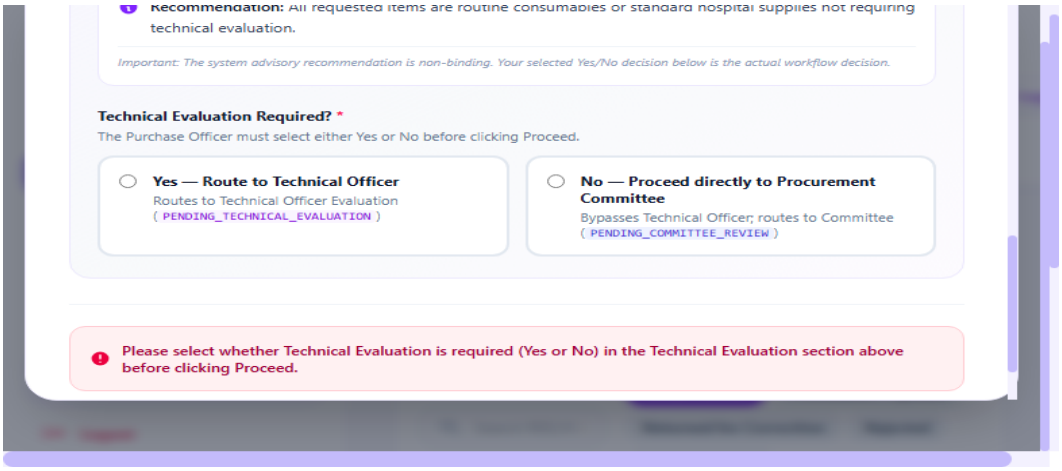


Figure 9 : Required Technical Routing Decision Validation Alert

Test Report
Test Case 3

Project Name: ProcuraMed	
Purchase Officer Request Review Test Case	
Test Case ID: Test_3	**Test Designed By:** QA Automation Engineer
Test Priority(Low/Medium/High): High	**Test Designed Date:** 22/10/2026
Module Name: Purchase Requisition Review	**Test Executed By :** QA Automation Engineer
Test Title : verify review of submitted purchase request, technical routing decision, and status progression.	**Test Execution Date:** 22/10/2026
Description: Purchase Officer logs in, reviews pending requisition specifications and budget, selects required technical evaluation routing, enters verification remarks, and proceeds requisition.	
Pre-Condition : Authorized Purchase Officer user is logged into ProcuraMed portal and has pending submitted purchase requests in their queue.	

Step	Test Step	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Log in as Purchase Officer and navigate to Requisitions Review page	URL: /purchase-officer/requisitions User: test_po	Display pending purchase requisitions assigned to the Purchase Officer	Pending requisitions table displayed with request tracking numbers, department, and Submitted status	**pass**
2	Select pending requisition and open Review modal	Requisition: REQ-2026-00004 Title: Automated Test - Patient Monitor Review	Open Review modal displaying requisition details, line items, and action panel	Review modal opened showing line items, budget (■45,000.00), and clinical justification	**pass**
3	Select Technical Evaluation routing decision	Decision: YES Category: Biomedical Equipment	Technical evaluation routing decision accepted; routing advisory updated	Technical Evaluation YES selected; advisory directs request to Biomedical Engineering	**pass**
4	Initiate Proceed action, enter verification remarks, and confirm	Remarks: 'Technical specifications and clinical budget verified. Proceeding for Technical Officer evaluation.'	Action modal opens, remarks submitted, confirmation message displayed: 'Requisition proceeded successfully'	Confirmation banner displayed: 'Requisition proceeded successfully'; review modal closes	**pass**
5	Navigate to Proceeded tab and verify updated requisition status	Tab: Proceeded Requisition: REQ-2026-00004	Requisition status updated from 'SUBMITTED' to 'Pending Technical Evaluation'	Requisition displayed under Proceeded tab with updated status badge 'Pending Technical Evaluation'	**pass**

Post-Condition: Requisition status updated to 'PENDING_TECHNICAL_EVALUATION' in database, audit log entry created, and requisition routed to Technical Officer queue.

Test Report – Negative Test Cases
Test Case 3.1: Technical Routing Validation

Project Name: ProcuraMed					
Purchase Officer Review Routing Validation Test Cases					
Test Case ID: Test_3_Neg			**Test Designed By:** QA Automation Engineer		
Test Priority(Low/Medium/High): Medium			**Test Designed Date:** 22/10/2026		
Module Name: Purchase Requisition Review			**Test Executed By :** QA Automation Engineer		
Test Title : verify validation error when proceeding without selecting technical evaluation routing decision.			**Test Execution Date:** 22/10/2026		
Description: Verify system enforces mandatory technical evaluation determination (Yes/No) before permitting requisition to proceed.					
Pre-Condition : Purchase Officer has opened review modal for a submitted requisition.					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Open review modal for pending submitted requisition	Requisition: REQ-2026-00004	Display Review Purchase Requisition modal	Review modal displayed with routing section unselected	**pass**
2	Attempt clicking 'Proceed' without selecting Technical Evaluation Yes/No	Routing Decision: [Unselected]	Validation error banner displayed: 'Please select whether Technical Evaluation is required (Yes or No) in the Technical Evaluation section above before clicking Proceed.'	Validation error banner displayed; proceed action blocked; status unchanged	**pass**
Post-Condition: Review action is blocked; requisition status remains 'SUBMITTED'; data integrity preserved.					